

מס' שאלון - 468
בספטמבר 2023

מס' מועד 99

סמסטר 2023ב

20441 / 4

שאלון בחינת גמר

20441 - מבוא למדעי המחשב ושפת Java

משך בחינה: 3 שעות

בשאלון זה 16 עמודים

מבנה הבחינה:

- קראו בעיון את ההנחיות שלהלן:
- * בבחינה יש חמש שאלות. עליכם לענות על כולן.
 - * כל התכניות צריכות להיות מתועדות היטב.
 - יש לכתוב תחילה בקצרה את האלגוריתם וכל הסבר נוסף הדרוש להבנת התכנית.
 - יש לבחור בשמות משמעותיים למשתנים, לפונקציות ולקבועים שבתכנית.
 - תכנית שלא תתועד כנדרש לעיל תקבל לכל היותר 85 % מהניקוד.
 - * יש להקפיד לכתוב את התכניות בצורה מבנית ויעילה.
 - תכנית לא יעילה לא תקבל את מלוא הנקודות.
 - * אם ברצונכם להשתמש בתשובתכם בשיטה או במחלקה הכתובה בחוברת השקפים, אין צורך שתעתיקו את השיטה או את המחלקה למחברת הבחינה.
 - מספיק להפנות למקום הנכון,
 - ובלבד שההפניה תהיה מדויקת (פרמטרים, מיקום וכו').
 - * אין להשתמש במחלקות קיימות ב-Java, חוץ מאלו המפורטות בשאלות הבחינה.
 - * יש לשמור על סדר; תכנית הכתובה בצורה בלתי מסודרת עלולה לגרוע מהציון.
 - * בכתיבת התכניות יש להשתמש אך ורק במרכיבי השפה שנלמדו בקורס זה
 - אין להשתמש במשתנים גלובליים!
 - * אפשר לתעד בעברית. אין צורך בתיעוד API.
 - * על שאלות 3-5 יש לענות אך ורק בשאלון ולא במחברת הבחינה!

חומר עזר:

חוברות השקפים 1-6, 7-12. אסור לכתוב כלום בתוך חוברות השקפים.
מותר לסמן עמודים בצבע או בדגלונים.
אין להכניס מחשב או מחשבון או מכשיר אלקטרוני מכל סוג שהוא.
אין להכניס חומר נוסף אחר מכל סוג.

החזירו

למשגיח את השאלון

וכל עזר אחר שקיבלתם בתוך מחברת התשובות

בהצלחה !!!



חלק א – עליכם לענות על כל השאלות בחלק זה במחברת הבחינה

שאלה 1 (25 נקודות)

פלינדרום (Palindrome) הוא מילה, מספר, משפט או כל רצף סמלים אחר שקריאתו מימין לשמאל ומשמאל לימין היא זהה.

לדוגמא: מחרוזות התווים "aba", "1221" הן פלינדרום.

במערך חד-ממדי המכיל מספרים שלמים, נגדיר **רצף פלינדרומי** כסדרה של תאים רצופים במערך המהווים פלינדרום.

נגדיר: רצף כמעט-פלינדרומי (Almost Palindrome) הוא רצף מספרים במערך חד ממדי כך שאם נשמיט ממנו ערך אחד **לכל היותר** (בתוך הרצף ולא בקצה) נקבל רצף פלינדרומי.

שימו לב שכל רצף פלינדרומי הוא גם רצף כמעט-פלינדרומי, כי לא משמיטים שום ערך ובכל זאת מקבלים רצף פלינדרומי.

לדוגמא:

במערך arr הבא:

0	1	2	(3)	4	5	6	7	(8)
1	1	4	10	10	4	3	10	10

ישנם שישה רצפים כמעט-פלינדרומיים:

1. בין האינדקסים 0 ל-1 {1, 1} זהו פלינדרום באורך 2 ולכן הוא גם כמעט-פלינדרום
2. בין האינדקסים 3 ל-4 {10, 10} זהו פלינדרום באורך 2 ולכן הוא גם כמעט-פלינדרום
3. בין האינדקסים 7 ל-8 {10, 10} זהו פלינדרום באורך 2 ולכן הוא גם כמעט-פלינדרום
4. בין האינדקסים 2 ל-5 {4, 10, 10, 4} זהו פלינדרום באורך 4 ולכן זה כמעט-פלינדרום
5. בין האינדקסים 4 ל-7 {10, 4, 3, 10} זהו כמעט-פלינדרום שאורכו 4.
כי אם נוריד את 4 או את 3 נקבל פלינדרום.
6. בין האינדקסים 3 ל-8 {10, 10, 4, 3, 10, 10} זהו כמעט-פלינדרום שאורכו 6.
כי אם נוריד את 4 או את 3 נקבל פלינדרום.

ועוד תשעה רצפים כמעט-פלינדרומיים, כל אחד מהם באורך 1, שהם כל אחד מהתאים (כל אחד מהם הוא פלינדרום, ולכן הוא כמעט-פלינדרום).

שימו לב שהרצף בין האינדקסים 2 ל-6 {4, 10, 10, 4, 3} אינו כמעט-פלינדרום כי הערך שהתווסף לפלינדרום {4, 10, 10, 4} נמצא **בקצה הרצף ולא בתוכו**.

כתבו שיטה סטטית רקורסיבית, המקבלת כפרמטר מערך חד-ממדי arr המכיל מספרים שלמים ומחזירה את אורכו של הרצף הכמעט-פלינדרומי הגדול ביותר.

לדוגמא, במערך arr שלעיל, הרצף הכמעט-פלינדרומי הגדול ביותר הוא בין האינדקסים 3 ל-8, ואורכו הוא 6. לכן, השיטה צריכה להחזיר את הערך 6. על המערך $arr = \{1, 2, 3, 4\}$ השיטה תחזיר 1.

חתימת השיטה היא:

```
public static int longestAlmostPalindrome (int[] arr)
```

השיטה צריכה להיות רקורסיבית ללא שימוש בלולאות כלל. כך גם כל שיטות העזר שתכתבו (אם תכתבו) לא יכולות להכיל לולאות.

מותר להשתמש בהעמסת-יתר (overloading).

אין צורך לדאוג ליעילות השיטה, אבל כמובן שצריך לשים לב לא לעשות קריאות רקורסיביות מיותרות!

אסור לשנות את המערך, אפילו לא באופן זמני!

אסור להשתמש במשתנים סטטיים (גלובליים)!

מותר להשתמש בשיטות Math.max ו-Math.min מהמחלקה Math וכן בקבועים Integer.MIN_VALUE, Integer.MAX_VALUE מהמחלקה Integer.

אל תשכחו לתעד את מה שכתבתם!

שאלה 2 (25 נקודות)

נתון מערך חד-ממדי arr המלא במספרים שלמים **שונים זה מזה**, והוא ממוין בסדר יורד.

כתבו שיטה סטטית המקבלת כפרמטר מערך arr כזה, ומוצאת מספר k שהוא גדול מהערך הקטן ביותר במערך, קטן מהערך הגדול ביותר במערך, **ולא נמצא במערך**.

לדוגמא,

המערך הבא הוא מערך ממוין.

0	1	2	3	4	5	6
80	70	50	40	20	10	3

קיימים מספרים רבים שגדולים מ-3 וקטנים מ-80 ולא נמצאים במערך, למשל, 4, 15, 21, 79 ועוד ועוד. השיטה צריכה להחזיר אחד מהם. לא משנה מי.

אם לא קיים מספר כזה, השיטה תחזיר את הערך Integer.MIN_VALUE (הערך השלם הקטן ביותר האפשרי).

אפשר להניח שגודל המערך הוא לפחות 2.

כמו כן, אפשר להניח שהמערך ממוין בסדר יורד ואין צורך לבדוק זאת!

חתימת השיטה היא:

```
public static int findNumber (int[] arr)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

ציינו מהי סיבוכיות זמן הריצה ומהי סיבוכיות המקום של השיטה שכתבתם. הסבירו תשובתכם.

אל תשכחו לתעד את מה שכתבתם!

חלק ב - את התשובות לשאלות 3, 4 ו- 5 יש לכתוב על גבי השאלון. לא נבדוק תשובות שייכתבו במקום אחר!

שאלה 3 (16 נקודות)

נניח שהמחלקה Node שלהלן מממשת צומת בעץ בינרי.

```
public class Node
{
    private int _number;
    private Node _leftSon, _rightSon;

    public Node (int number) {
        _number = number;
        _leftSon = null;
        _rightSon = null;
    }

    public int  getNumber()      {return _number; }
    public Node getLeftSon()     {return _leftSon; }
    public Node getRightSon()    {return _rightSon; }
}
```

המחלקה BinaryTree מאגדת בתוכה שיטות סטטיות לטיפול **בעץ בינרי**.

בין השיטות נתונות השיטות secret ו- what הבאות:

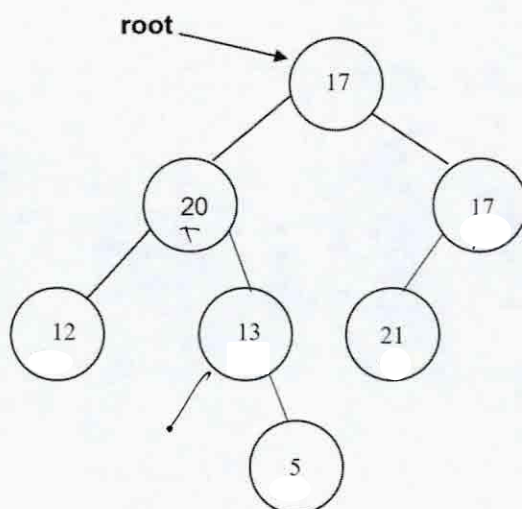
```
public static int secret(Node t)
{
    int val = t.getNumber();
    if (t.getLeftSon() == null && t.getRightSon() == null)
        return val;
    if (t.getLeftSon() != null)
        val = Math.max(val , secret(t.getLeftSon()));
    if (t.getRightSon() != null)
        val = Math.max(val , secret(t.getRightSon()));
    return val;
}
```

```

public static boolean what(Node t)
{
    if (t.getLeftSon() == null || t.getRightSon() == null)
        return true;
    if (secret(t.getLeftSon()) > secret(t.getRightSon()))
        return false;
    return what(t.getLeftSon()) && what(t.getRightSon());
}

```

נתון העץ הבינרי הבא, ששורשו הוא root



ענו על הסעיפים הבאים:

(א) מה תחזיר השיטה **secret** בעקבות הקריאה `BinaryTree.secret(root)`? (2 נק')
 התשובה היא:

(ב) מהם השינויים המינימליים שצריך לעשות על העץ לעיל, כדי שהקריאה לשיטה `secret(root)` תחזיר את הערך 20? שימו לב, השינוי צריך להיות **בעץ ולא בשיטה** (כגון, הוספת צומת, הורדת צומת, שינוי ערך של צומת וכד'). מינימלי במובן של מינימום פעולות על העץ. כך, אם מורידים צומת, ויש לצומת הזה צאצאים, במספר הפעולות נספרים גם הצאצאים של הצומת שהורדו. **עליכם לכתוב כמה שינויים צריך לעשות ומהם השינויים האלו ולנמק את תשובתכם.** (2 נק')

התשובה היא:

(ג) מה מבצעת השיטה **secret** באופן כללי כשהיא מקבלת כפרמטר שורש של עץ בינרי **root**? שימו לב, עליכם לתת תיאור ממצה של מה עושה השיטה באופן כללי, ולא תיאור של מה עושה כל שורה בשיטה, או איך היא מבצעת זאת. כלומר, כתבו בקצרה מה משמעות הערך שהשיטה מחזירה כשהיא מקבלת שורש של עץ. (3 נק')

התשובה היא:

(ד) מה תחזיר השיטה **what** בעקבות הקריאה `BinaryTree.what(root)`? (2 נק')

התשובה היא:

(ה) מה מבצעת השיטה **what** באופן כללי כשהיא מקבלת כפרמטר שורש של עץ בינרי **root**? שימו לב, עליכם לתת תיאור ממצה של מה עושה השיטה באופן כללי, ולא תיאור של מה עושה כל שורה בשיטה, או איך היא מבצעת זאת. כלומר, כתבו בקצרה מה משמעות הערך שהשיטה מחזירה כשהיא מקבלת שורש של עץ. (3 נק')

התשובה היא:

(ו) אנו רוצים לשנות את הערך של הצומת 13, אבל לדאוג לכך שהתוצאה של הקריאה `what(root)` לא תשתנה. מה טווח הערכים שניתן לשים במקום הצומת 13? (2 נק')

התשובה היא:

(ז) האם קיים עץ **חיפוש בינרי** (עץ ממזין), בן לפחות 5 צמתים, עבורו השיטה **what** תחזיר `false`? אם כן, ציירו עץ כזה. אם לא, הסבירו מדוע אין כזה. (2 נק')

התשובה היא:

נתון פרויקט שהוגדרו בו המחלקות A, B, C, D ו-Driver שלהלן. כל אחת בקובץ נפרד, כמובן.

```
public class A {
    public boolean equals(Object obj) {
        System.out.println("1");
        return super.equals(obj);
    }
} //end of class A
```

```
-----
public class B extends A {
    public boolean equals(Object obj) {
        System.out.println("2");
        return super.equals(obj);
    }

    public boolean equals(B obj) {
        System.out.println("3");
        return super.equals(obj);
    }
} //end of class B
```

```
-----
public class C extends A {
    public boolean equals(A obj) {
        System.out.println("4");
        return super.equals(obj);
    }
} //end of class C
```

```
-----
public class D extends C {
    public boolean equals(A obj) {
        System.out.println("5");
        return super.equals(obj);
    }

    public boolean equals(B obj) {
        System.out.println("6");
        return super.equals(obj);
    }
} //end of class D
-----
```



```

public class Driver
{
    public static void main (String [] args)
    {
        A a1 = new A();
        B b1 = new B();
        C c1 = new C();
        D d1 = new D();

        Object a2 = new A();
        A b2 = new B();
        A c2 = new C();
        D d2 = new D();
        A d3 = new D();

        // כאן יופיעו הפקודות שבסעיפים להלן
    }
} //end of class Driver

```

עליכם לענות על כל הסעיפים הבאים.

אין תלות בין הסעיפים.

בכל סעיף עליכם לכתוב מה יקרה כתוצאה מהפעלת קטע הקוד. אם תהיה שגיאת קומפילציה או הרצה, כתבו איזו שגיאה ונמקו. אם הקוד תקין, כתבו מה יודפס על הפלט.

שימו לב לדייק בהדפסה!

לא בהכרח כל השורות יתמלאו!

סעיף א (2 נקודות) –

נניח שהפקודה היא:

b1.equals(d1);

התשובה היא:

_____	_____
_____	_____
_____	_____

סעיף ב (2 נקודות) –

נניח שהפקודה היא:

```
a2.equals((D)b1);
```

התשובה היא:

סעיף ג (2 נקודות) –

נניח שהפקודה היא:

```
d1.equals(b1);
```

התשובה היא:

סעיף ד (2 נקודות) –

נניח שהפקודה היא:

```
d1.equals(b2);
```

התשובה היא:

סעיף ה (2 נקודות) –

נניח שהפקודה היא:

```
c1.equals(b2);
```

התשובה היא:

סעיף ו (2 נקודות) –
נניח שהפקודה היא:

```
a2.equals((D)c1);
```

התשובה היא:

סעיף ז (2 נקודות) –
נניח שהפקודה היא:

```
a2.equals((Object)c1);
```

התשובה היא:

סעיף ח (2 נקודות) –

איזו פקודה צריך לכתוב ב- main כדי שמה שיודפס על הפלט יהיה (בשורות נפרדות)

3

1

שימו לב שהכוונה היא לכתוב פקודה כמו אלו שהיו בסעיפים הקודמים, ולהשתמש אך ורק במשתנים שמוגדרים כבר ב- main. לא להצהיר על משתנים חדשים.

התשובה היא:

שאלה 5 (18 נקודות)

נתונה המחלקה IntNodeTwo הבאה, המייצגת איבר ברשימה מקושרת דו-סטטית המכילה מספרים שלמים:

```
public class IntNodeTwo
{
    private int _num;
    private IntNodeTwo _next, _prev;

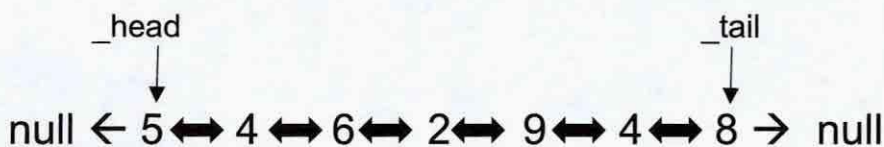
    public IntNodeTwo(int n)
    {
        _num = n;
        _next = null;
        _prev = null;
    }

    public IntNodeTwo(int num, IntNodeTwo n, IntNodeTwo p)
    {
        _num = num;
        _next = n;
        _prev = p;
    }

    public int getNum() { return _num; }
    public IntNodeTwo getNext() { return _next; }
    public IntNodeTwo getPrev() { return _prev; }
    public void setNum (int n) { _num = n; }
    public void setNext (IntNodeTwo node) { _next = node; }
    public void setPrev (IntNodeTwo node) { _prev = node; }
}
```

נתונה רשימה מקושרת דו-סטטית, הממומשת בעזרת המחלקה IntListTwo שלהלן.

נתונה הרשימה list הבאה:



לשם קיצור, נכתוב את הרשימה שלעיל כך, כאשר המספר משמאל הוא ראש הרשימה _head והמספר מימין הוא זנב הרשימה _tail:

{5, 4, 6, 2, 9, 4, 8}

```
public class IntListTwo
{
    private IntNodeTwo _head, _tail;

    public IntListTwo()
    {
        _head = null;
        _tail = null;
    }

    public IntListTwo(IntNodeTwo node)
    {
        _head = node;
        _tail = node;
        while (_tail.getNext() != null)
            _tail = _tail.getNext();
    }

    public IntListTwo something (IntNodeTwo p)
    {
        int num = p.getNum();
        IntNodeTwo head = null, tail = null, ptr = null;
        ptr = new IntNodeTwo(num%10, tail, head);
        head = ptr;
        tail = ptr;
        num = num/10;
        while (num > 0)
        {
            ptr = new IntNodeTwo(num%10, head, null);
            head.setPrev(ptr);
            head = ptr;
            num = num/10;
        }
        return new IntListTwo(head);
    } //end of something

    // המשך המחלקה בעמוד הבא //
}
```

```

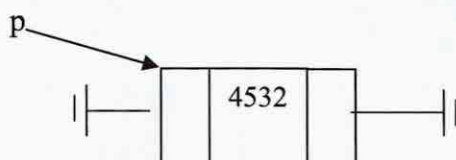
public IntListTwo secret()
{
    IntNodeTwo ptr = _head;
    IntNodeTwo p1 = null, p2 = new IntNodeTwo(0);
    while (ptr != null)
    {
        IntListTwo curr = something(ptr);
        if (p1 == null)
            p1 = curr._head;
        p2.setNext(curr._head);
        p2 = curr._tail;
        ptr = ptr.getNext();
    }
    return new IntListTwo(p1);
} //end of secret

} // end of class IntListTwo

```

סעיף א (2 נקודות)

מה תחזיר השיטה **something** אם נעביר לה כפרמטר את המצביע p לאיבר הבא (איבר שהשדות $_next$ ו- $_prev$ שלו הם null):



התשובה היא:

סעיף ב (3 נקודות)

מה מבצעת השיטה **something** באופן כללי? הסבירו בקצרה מה השיטה עושה ולא כיצד היא מבצעת זאת.

שימו לב, עליכם לתת תיאור ממצה של מה עושה השיטה באופן כללי, ולא תיאור של מה עושה כל שורה בשיטה, או איך היא מבצעת זאת. הסבירו מה משמעותו של הערך המוחזר מהשיטה **something**? התייחסו למקרי קצה.

התשובה היא:

נשנה את השיטה something לשיטה anotherSomething כך:

```
public IntListTwo anotherSomething (IntNodeTwo p)
{
    int num = p.getNum();
    IntNodeTwo head = null, tail = null, ptr = null;
    ptr = new IntNodeTwo(num%10, tail, head);
    head = ptr;
    tail = ptr;
    num = num/10;
    while (num > 0)
    {
        ptr = new IntNodeTwo(num%10, null, tail);
        tail.setNext(ptr);
        tail = ptr;
        num = num/10;
    }
    return new IntListTwo(head);
}
```

סעיף ג (2 נקודות)

מה צריך להיות ערכו של הפרמטר p, כדי שאם נפעיל את השיטה anotherSomething עם הפרמטר p, הרשימה שהשיטה תחזיר תהיה:

{ 1 , 2 , 3 , 4 }

התשובה היא:

סעיף ד (3 נקודות)

מה מבצעת השיטה anotherSomething באופן כללי? הסבירו בקצרה מה השיטה עושה ולא כיצד היא מבצעת זאת.

שימו לב, עליכם לתת תיאור ממצה של מה עושה השיטה באופן כללי, ולא תיאור של מה עושה כל שורה בשיטה, או איך היא מבצעת זאת. הסבירו מה משמעותו של הערך המוחזר מהשיטה anotherSomething? התייחסו למקרי קצה.

התשובה היא:

סעיף ה (4 נקודות)

נתונה הרשימה list הבאה:

{428 , 354 , 78 , 8545}

אם נפעיל את השיטה secret על הרשימה list לעיל, מה תחזיר השיטה?

התשובה היא:

סעיף ו (4 נקודות)

אם נשנה את השיטה secret כך שבמקום הקריאה לשיטה something בפקודה

```
IntListTwo curr = something(ptr);
```

תיקרא השיטה anotherSomething כך:

```
IntListTwo curr = anotherSomething(ptr);
```

ונפעיל את השיטה secret (המעודכנת) על הרשימה list לעיל, מה תחזיר השיטה?

התשובה היא:

ב ה צ ל ח !

